

Pontificia Universidad Javeriana
Computación de alto desempeño
Taller Clúster Hadoop HDFS

John Corredor, PhD

Integrantes	Juan Sebastián Bravo Santacruz – bravos.js@javeriana.edu.co Felix David Cordova Garcia – fdcordova@javeriana.edu.co Jonatan Alejandro Gallo Martinez – jonatan.gallo@javeriana.edu.co Jose Alejandro Jaime Lopez – jo.jaime@javeriana.edu.co Josman Alexdy Ramirez Torres – josman.ramirez@javeriana.edu.co Jesús David Romero Melo – jesus-romero@javeriana.edu.co Juan Camilo Torres Peña – torrespjc@javeriana.edu.co
Fecha	2026/05/21

I. Introducción

Hadoop Distributed File System (HDFS) es un sistema de almacenamiento distribuido diseñado para manejar grandes volúmenes de datos de manera tolerante a fallos. Su diseño está inspirado en el Google File System (GFS), propuesto por Ghemawat, Gobioff y Leung (2003), el cual introdujo la arquitectura maestra/esclavo con replicación de datos en múltiples nodos para garantizar disponibilidad y rendimiento en clústeres de hardware convencional.

HDFS fue desarrollado como una implementación de código abierto de estos principios por ingenieros de Yahoo, y su arquitectura fue documentada formalmente por Shvachko, Kuang, Radia y Chansler (2010). El sistema utiliza un NameNode como servidor maestro que gestiona el espacio de nombres, y múltiples DataNodes que almacenan los bloques de datos replicados. Este modelo, combinado con el paradigma MapReduce propuesto por Dean y Ghemawat (2004), constituye la base del ecosistema Apache Hadoop para el procesamiento de alto volumen de datos (Apache Software Foundation, s.f.).

En este taller se implementó un clúster HDFS compuesto por un NameNode y varios DataNodes utilizando máquinas virtuales Linux Rocky, realizando la configuración completa del entorno Hadoop y la validación del sistema mediante comandos de administración y transferencia de archivos.

II. Contexto

En el laboratorio anterior se configuró un clúster Apache Spark 3.5.2 sobre máquinas virtuales Rocky Linux, compuesto por un nodo maestro (cadhead02) y cinco workers, utilizando NFS como sistema de archivos compartido para distribuir los binarios de Spark desde el servidor cad02-nfs01 hacia todos los nodos mediante el directorio /nfs/condor. Si bien esta configuración resultó funcional para garantizar consistencia de versiones y simplificar el despliegue, NFS presenta limitaciones importantes como capa de almacenamiento de datos: depende de un único servidor (cad02-nfs01) que constituye un punto único de falla, no ofrece replicación nativa de los datos almacenados, y su rendimiento se degrada cuando múltiples nodos acceden concurrentemente a archivos de gran tamaño (White, 2015). Estas restricciones son relevantes en escenarios de computación de alto desempeño donde Spark necesita leer datasets distribuidos de forma paralela. HDFS fue diseñado específicamente para resolver estas limitaciones: su arquitectura divide los archivos en bloques que se replican y distribuyen automáticamente entre los DataNodes, eliminando el punto único de falla y permitiendo lecturas paralelas desde múltiples nodos simultáneamente (Shvachko et al., 2010). Además, Spark integra soporte nativo para leer y escribir sobre HDFS, lo que permite evaluar si reemplazar NFS por HDFS como capa de almacenamiento mejora el rendimiento del clúster existente. El presente taller implementa un clúster Hadoop HDFS sobre la misma infraestructura Rocky Linux y realiza experimentos comparativos entre ambos enfoques.

III. Objetivo

Implementar y configurar un clúster Hadoop HDFS funcional para almacenamiento distribuido de datos sobre máquinas virtuales Linux Rocky, aplicando los principios de sistemas de archivos distribuidos descritos en la literatura (Ghemawat et al., 2003; Shvachko et al., 2010).

IV. Configuración de cluster

A continuación, se evidencian los pasos realizados para el presente taller.

Paso 0: Definición de la Topología del Clúster

El clúster HDFS sigue una topología maestra/trabajador. El nodo Master aloja el NameNode, el cual tiene la función de gestionar los metadatos y supervisar el almacenamiento de los datos en el cluster, mientras que los nodos Workers alojan los DataNodes, responsables de crear, borrar y replicar bloques según las instrucciones del NameNode. La comunicación entre ellos se realiza por SSH.

Nombre de la Máquina	IP	Proceso
cadcliente02.javeriana.edu.co	10.43.97.145	NameNode
cadhead02.javeriana.edu.co	10.43.97.146	DataNode1
cad02-w003.javeriana.edu.co	10.43.97.148	DataNode2
cad02-w002.javeriana.edu.co	10.43.97.136	DataNode3
cad02-w001.javeriana.edu.co	10.43.97.135	DataNode4
cad02-w000.javeriana.edu.co	10.43.97.141	DataNode5
cad02-nfs01.javeriana.edu.co	10.43.97.149	NFS

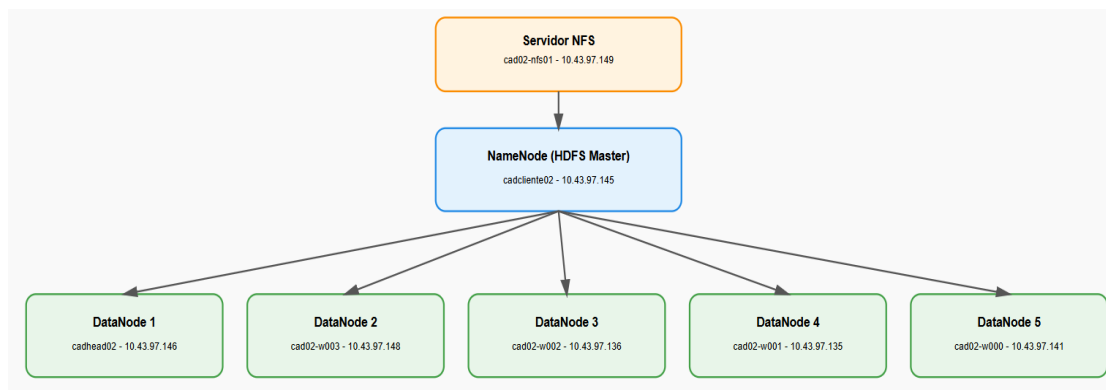


Figura 1. Diagrama arquitectura implementada

Paso 1: Configuración de Autenticación SSH sin Contraseña

Para que el NameNode pueda comunicarse con los DataNodes sin requerir contraseña, se genera un par de llaves SSH en el Master y se distribuye la llave pública a cada Worker.

Paso 2: Generar el par de claves SSH en el máster:

```
$ ssh-keygen
```

Paso 3: Copiar la clave pública al master y a cada worker:

```
$ ssh-copy-id master@<ip-del-master>
$ ssh-copy-id worker@<ip-del-worker>
```

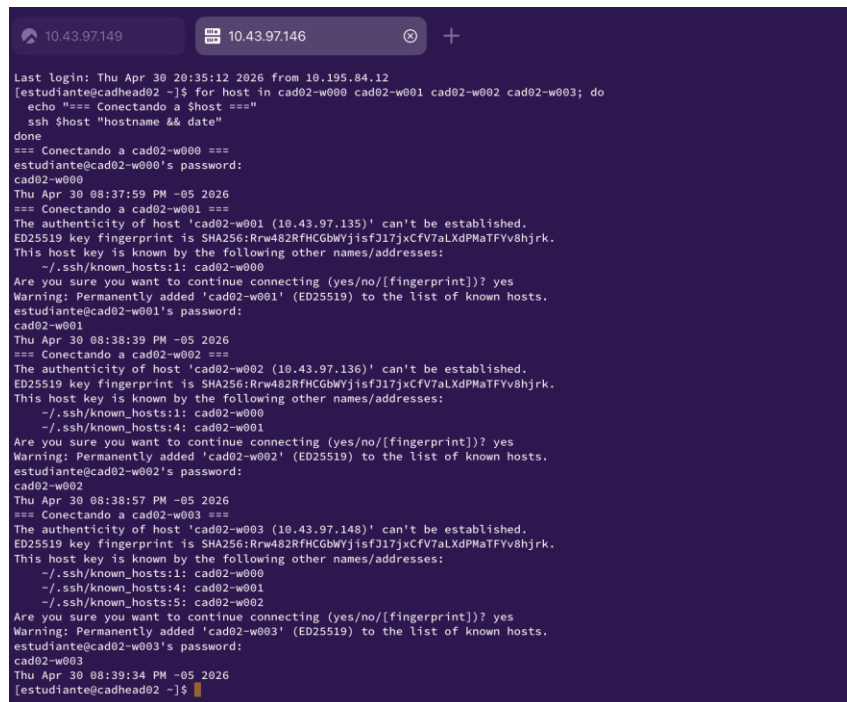
A terminal window with a dark purple background and white text. At the top, there are two tabs: the first is titled '10.43.97.149' and the second is titled '10.43.97.146'. The terminal shows a script being executed. The script starts with 'Last login: Thu Apr 30 20:35:12 2026 from 10.195.84.12' and then enters a loop: '[estudiante@cadhead02 ~]\$ for host in cad02-w000 cad02-w001 cad02-w002 cad02-w003; do echo "=== Conectando a \$host ===" ssh \$host "hostname && date" done'. The output shows the script connecting to each host. For each host, it displays the time, the user 'estudiante@cad02-w000', and the password prompt. It then shows the host's fingerprint and a warning: 'Warning: Permanently added \'cad02-w001\' (ED25519) to the list of known hosts.' The script continues to connect to cad02-w002 and cad02-w003, showing similar output for each.

Figura 2. Evidencia autenticación

Paso 4: Instalación Hadoop HDFS

Para iniciar la configuración del cluster, es necesario realizar los siguientes pasos, esto asegura que todos los nodos tengan la misma versión de Hadoop.

Se descarga la aplicación desde el repositorio de apache con curl.

```
$ curl -O https://dlcdn.apache.org/hadoop/common/hadoop-3.5.0/ hadoop-3.5.0.tar.gz
```

```
[estudiante@cadcliente02 hdfs]$ curl -O https://d1cdn.apache.org/hadoop/common/hadoop-3.5.0/ hadoop-3.5.0.tar.gz
% Total    % Received % Xferd Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left    Speed
100 554M  100 554M    0     0  163M      0  0:00:03  0:00:03 --:--:-- 163M
```

Se descomprime el archivo descargado dentro de la carpeta compartida por todos los nodos.

```
$ tar -xzf hadoop-3.5.0.tar.gz
```

```
[estudiante@cadcliente02 hdfs]$ tar -xzf hadoop-3.5.0.tar.gz
```

Se instala Java 17 en todos los nodos, ya que la última versión de Hadoop 3.5, funciona sobre esta versión de Java.

```
$ ./install_java_17.sh
```

```
● → high-performance-computing git:(main) X ./install_java_17.sh
Installing Java 17 on 10.43.97.141...
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Last metadata expiration check: 3:21:22 ago on Thu 14 May 2026 04:58:49 PM -05.
Dependencies resolved.

=====
Package                Arch      Version                               Repository  Size
=====
Installing:
java-17-openjdk         x86_64    1:17.0.19.0.10-2.el9_7               appstream  430 k
java-17-openjdk-devel   x86_64    1:17.0.19.0.10-2.el9_7               appstream  4.7 M
Installing dependencies:
java-17-openjdk-headless x86_64    1:17.0.19.0.10-2.el9_7               appstream  44 M
=====
Transaction Summary
=====
Install 3 Packages

Total download size: 49 M
Installed size: 196 M
Downloading Packages:
(1/3): java-17-openjdk-17.0.19.0.10-2.el9_7.x86_64 1.0 MB/s | 430 kB  00:00
(2/3): java-17-openjdk-devel-17.0.19.0.10-2.el9_7.x86_64 6.3 MB/s | 4.7 MB  00:00
(3/3): java-17-openjdk-headless-17.0.19.0.10-2.el9_7.x86_64 24 MB/s | 44 MB  00:01
-----
Total                                     21 MB/s | 49 MB  00:02
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Running scriptlet: java-17-openjdk-headless-1:17.0.19.0.10-2.el9_7.x86_64 1/1
  Preparing : 1/1
  Installing : java-17-openjdk-headless-1:17.0.19.0.10-2.el9_7.x86_64 1/3
  Running scriptlet: java-17-openjdk-headless-1:17.0.19.0.10-2.el9_7.x86_64 1/3
  Installing : java-17-openjdk-1:17.0.19.0.10-2.el9_7.x86_64 2/3
  Running scriptlet: java-17-openjdk-1:17.0.19.0.10-2.el9_7.x86_64 2/3
  Installing : java-17-openjdk-devel-1:17.0.19.0.10-2.el9_7.x86_64 3/3
  Running scriptlet: java-17-openjdk-devel-1:17.0.19.0.10-2.el9_7.x86_64 3/3
  Running scriptlet: java-17-openjdk-headless-1:17.0.19.0.10-2.el9_7.x86_64 3/3
  Running scriptlet: java-17-openjdk-1:17.0.19.0.10-2.el9_7.x86_64 3/3
  Running scriptlet: java-17-openjdk-devel-1:17.0.19.0.10-2.el9_7.x86_64 3/3
  Verifying : java-17-openjdk-1:17.0.19.0.10-2.el9_7.x86_64 1/3
  Verifying : java-17-openjdk-devel-1:17.0.19.0.10-2.el9_7.x86_64 2/3
  Verifying : java-17-openjdk-headless-1:17.0.19.0.10-2.el9_7.x86_64 3/3
=====
Installed:
java-17-openjdk-1:17.0.19.0.10-2.el9_7.x86_64
java-17-openjdk-devel-1:17.0.19.0.10-2.el9_7.x86_64
java-17-openjdk-headless-1:17.0.19.0.10-2.el9_7.x86_64
=====
Complete!
Installing Java 17 on 10.43.97.135...
** WARNING: connection is not using a post-quantum key exchange algorithm.
```

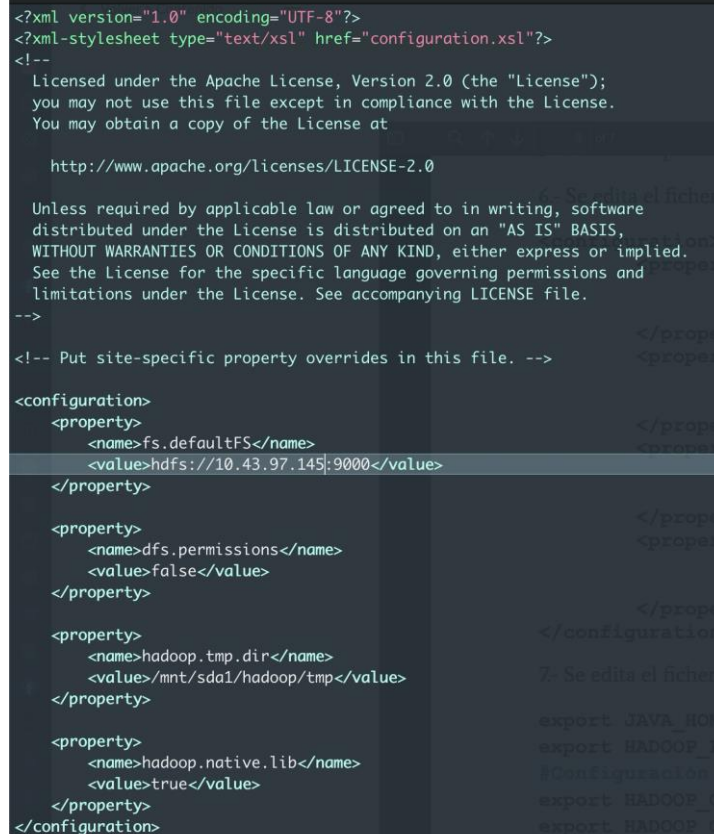
Figura 3. Evidencia instalación Java.

Paso 5: configuración Hadoop HDFS

Las siguientes configuraciones se realizan para definir el funcionamiento del clúster, establecer la comunicación entre el nodo maestro y los nodos de trabajo, y asegurar que todas las variables y dependencias del sistema estén correctamente configuradas en cada máquina.

Paso 5.1: Se edita el archivo de configuración core-site.xml indicando la dirección del nodo main

```
$ vi etc/hadoop/core-site.xml
```



```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License. See accompanying LICENSE file.
-->
<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://10.43.97.145:9000</value>
  </property>

  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>

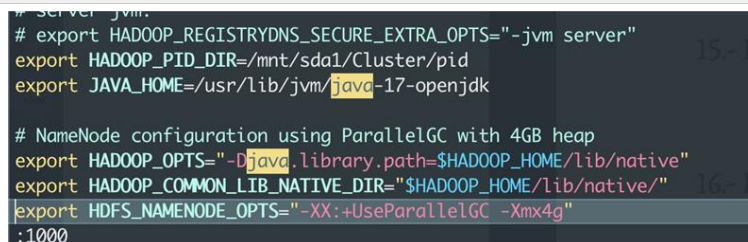
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/mnt/sda1/hadoop/tmp</value>
  </property>

  <property>
    <name>hadoop.native.lib</name>
    <value>true</value>
  </property>
</configuration>
```

Figura 4. Evidencia configuración core-site.xml.

Paso 5.2: Se edita el archivo que tiene las variables de entorno indicando el jdk de Java.

```
$ vi etc/hadoop/hadoop-env.sh
```



```
# Server java.
# export HADOOP_REGISTRYDNS_SECURE_EXTRA_OPTS="-jvm server"
export HADOOP_PID_DIR=/mnt/sda1/Cluster/pid
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk

# NameNode configuration using ParallelGC with 4GB heap
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
export HADOOP_COMMON_LIB_NATIVE_DIR="$HADOOP_HOME/lib/native/"
export HDFS_NAMENODE_OPTS="-XX:+UseParallelGC -Xmx4g"
:1000
```

Figura 5. Evidencia variables de entorno.

Paso 5.3: Se edita el archivo workers para indicar las direcciones de los data nodes siguiendo la siguiente tabla.

```
$ vi workers
```

Nombre de la Máquina	IP	Proceso
cadcliente02.javeriana.edu.co	10.43.97.145	NameNode
cadhead02.javeriana.edu.co	10.43.97.146	DataNode1
cad02-w003.javeriana.edu.co	10.43.97.148	DataNode2
cad02-w002.javeriana.edu.co	10.43.97.136	DataNode3
cad02-w001.javeriana.edu.co	10.43.97.135	DataNode4
cad02-w000.javeriana.edu.co	10.43.97.141	DataNode5
cad02-nfs01.javeriana.edu.co	10.43.97.149	NFS

Paso 5.4: Editamos el archivo hdfs-site.xml señalando la carpeta de almacenamiento de cada data node que estará disponible para Hadoop.

```
$ vi etc/hadoop/hdfs-site.xml
```

```
[estudiante@cadcliente02 hadoop-3.5.0]$ cat etc/hadoop/hdfs-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>

  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///mnt/HadoopHDFS/NameNode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///mnt/HadoopHDFS/DataNode</value>
  </property>

  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>

  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>

  <property>
    <name>dfs.webhdfs.enabled</name>
    <value>true</value>
  </property>

  <property>
    <name>ipc.maximum.data.length</name>
    <value>134217728</value>
  </property>

</configuration>
[estudiante@cadcliente02 hadoop-3.5.0]$
```

Figura 6. Evidencia archivo hdfs-site.xml

Paso 5.5: Se edita el archivo `/home/estudiante/.bashrc`, se añade la información del jdk de java, la ubicación del Spark y el Hadoop, las variables de configuración y el usuario del main y los data nodes.

```
$ vi /home/estudiante/.bashrc
```

```
unset rc

export SPARK_HOME=/nfs/condor/apps/Spark
export SPARK_CONF_DIR=$SPARK_HOME/conf
export PATH=$SPARK_HOME/bin:$PATH
export PYSPARK_PYTHON=/usr/bin/python3
export PYSPARK_DRIVER_PYTHON=/usr/bin/python3

export JAVA_HOME=/usr/lib/jvm/java-17-openjdk

export HADOOP_HOME=/nfs/condor/apps/hadoop-3.5.0
export HADOOP_INSTALL=$HADOOP_HOME

export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME

export HADOOP_CONF_DIR=${HADOOP_HOME}/etc/hadoop

export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HADOOP_HOME/lib/native

export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin

export HDFS_NAMENODE_USER="estudiante"
export HDFS_DATANODE_USER="estudiante"
export HDFS_SECONDARYNAMENODE_USER="estudiante"
```

Figura 7. Evidencia archivo .bashrc

Paso 6: Creación carpetas necesarias para Hadoop en los nodos

Este paso tiene como objetivo crear y asignar las rutas de almacenamiento que utilizará Hadoop en el nodo maestro y los nodos de trabajo, garantizando que cada componente del clúster disponga de un espacio dedicado para el manejo de datos y su correcta administración.

Paso 6.1: Se crea la carpeta NameNode en el main node

```
$ mkdir /mnt/HadoopHDFS
$ sudo chown -R estudiante:estudiante /mnt/HadoopHDFS
```

```
[estudiante@cadcliente02 hadoop-3.5.0]$ sudo chown -R estudiante:estudiante /mnt/HadoopHDFS
```

Figura 8. Evidencia owner de la carpeta.

Paso 6.2: Se crea la carpeta NameNode en los data nodes

Se crean las carpetas *DataNode* de los data nodes en /mnt/HadoopHDFS/DataNode, Se cambia el propietario de la carpeta y los permisos. Para esto se creó un script que hiciera esta tarea en todos los workers.

```
hadoop_order_workers.sh
1  for HOST in 10.43.97.141 10.43.97.135 10.43.97.136 10.43.97.148 10.43.97.146 10.43.97.145
2
3      echo "Configuring Hadoop folders on $HOST..."
4
5      ssh estudiante@$HOST "
6          sudo mkdir -p /mnt/HadoopHDFS/DataNode && \
7          sudo chown -R estudiante:estudiante /mnt/HadoopHDFS
8          sudo chmod -R 777 /mnt/HadoopHDFS/DataNode
9      "
10 done
```

Figura 9. Script para creación de carpetas en data nodes.

Paso 7: Formateo de carpeta Hadoop

```
$ $HADOOP_HOME/bin/hdfs namenode -format
```

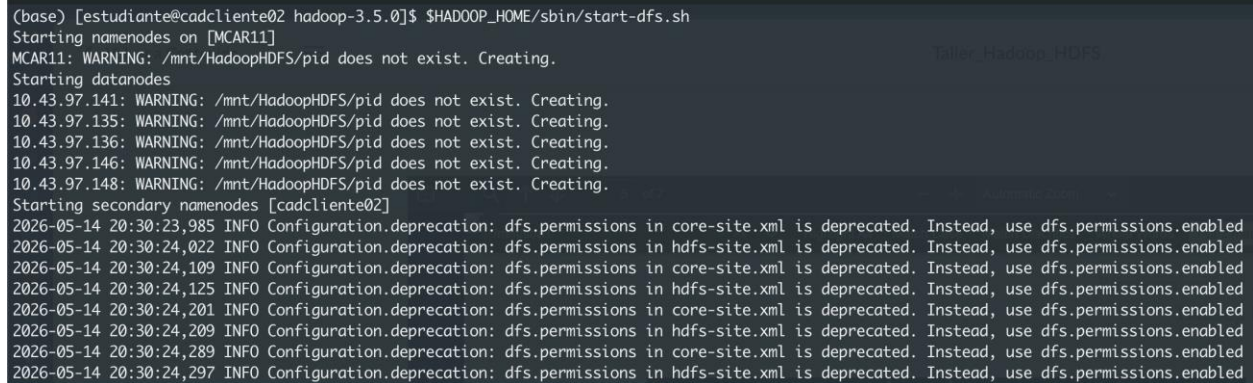
Desde el main node se corre el comando *namenode -format* para inicializar y formatear la carpeta main de Hadoop

```
(base) [estudiante@cadcliente02 ~]$ ssh $HADOOP_HOME/etc/hadoop/hadoop-env.sh
(base) [estudiante@cadcliente02 ~]$ $HADOOP_HOME/bin/hdfs namenode -format
2026-05-14 20:14:57.665 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG: host = cadcliente02/10.43.97.145
STARTUP_MSG: args = [-format]
STARTUP_MSG: version = 3.5.0
STARTUP_MSG: classpath = /nfs/condor/apps/hadoop-3.5.0/etc/hadoop:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/httpcore-4.4.13.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/commons-math3-3.6.1.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jakarta.v
mmons-codec-1.15.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/commons-configuration2-2.10.1.jar:/nfs/condor/apps/hadoop-3.5.0/sh
0/share/hadoop/common/lib/jetty-webapp-9.4.58.v20250814.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/bcprov-jdk18on-1.82.jar:/n
/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/javax.servlet-api-3.1.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/netty-tra
hadoop-3.5.0/share/hadoop/common/lib/jetty-server-9.4.58.v20250814.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/
j-reload4j-1.7.36.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/commons-cli-1.9.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/
karta.servlet-api-4.0.4.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/gson-2.9.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/c
hadoop-3.5.0/share/hadoop-gcp-3.5.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/reload4j-1.2.22.jar:/nfs/condor/apps/hadoop-3.5.
p-3.5.0/share/hadoop/common/lib/commons-lang3-3.18.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/snappy-java-1.1.10.4.jar:/nfs/
/hadoop-3.5.0/share/hadoop/common/lib/netty-resolver-4.1.130.Final.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/avro-1.11.5.jar:/
ondor/apps/hadoop-3.5.0/share/hadoop/common/lib/jakarta.activation-1.2.2.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/nimbus-jos
8-jre.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/hk2-locator-2.6.1.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/h
ce-annotations-0.12.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jsch-0.1.55.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/co
b/kerby-config-2.0.3.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jackson-annotations-2.18.6.jar:/nfs/condor/apps/hadoop-3.5.0/s
/share/hadoop/common/lib/jaxb-runtime-2.3.9.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/hadoop-annotations-3.5.0.jar:/nfs/condo
/hadoop-3.5.0/share/hadoop/common/lib/jetty-io-9.4.58.v20250814.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/dnsjava-3.6.1.jar:/
ondor/apps/hadoop-3.5.0/share/hadoop/common/lib/jersey-client-2.46.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/HdrHistogram-2.1
999.0-empty-to-avoid-conflict-with-guava.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jackson-core-2.18.6.jar:/nfs/condor/apps/h
oop-3.5.0/share/hadoop/common/lib/commons-io-2.16.1.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jakarta.annotation-api-1.3.5.ja
r:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jzobjc-annotations-3.0.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/j
common/lib/commons-logging-1.3.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/stax2-api-4.2.1.jar:/nfs/condor/apps/hadoop-3.5.0/
p-3.5.0/share/hadoop/common/lib/hadoop-azure-datalake-3.5.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/woodstox-core-5.4.0.jar
s/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jakarta.inject-2.6.1.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/commons-com
wei-cloud-3.5.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/slf4j-api-1.7.36.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/com
r/apps/hadoop-3.5.0/share/hadoop/common/lib/commons-daemon-1.0.13.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jetty-util-9.4.58.v20250814.jar:/nfs/condor/apps/hadoo
r/apps/hadoop-3.5.0/share/hadoop/common/lib/jackson-databind-2.18.6.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/httpclient-4.5
2.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/netty-transport-4.1.130.Final.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common
0/share/hadoop/common/lib/netty-handler-4.1.130.Final.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/osgi-resource-locator-1.0.3.j
s/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/azure-keyvault-core-1.0.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/hk2-ut
ns-runtime-3.0.12.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/azure-storage-7.0.1.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoo
adadoop-3.5.0/share/hadoop/common/lib/jetty-server-9.4.58.v20250814.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/wildfly-openssl-2
2.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/jersey-common-2.46.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/hado
net-3.9.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/zookeeper-jute-3.8.6.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/comma
hadoop-cos-3.5.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/netty-transport-classes-epoll-4.1.130.Final.jar:/nfs/condor/apps/h
nfs/hadoop-3.5.0/share/hadoop/common/lib/jakarta.xml.bind-api-2.3.3.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/lib/hk2-api-2.6.1.ja
r:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/hadoop-registry-3.5.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/hadoop-nfs-3.
nfs/condor/apps/hadoop-3.5.0/share/hadoop/common/hadoop-common-3.5.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/hdfs:/nfs/condor/apps/had
0/share/hadoop/hdfs/lib/curator-client-5.2.0.jar:/nfs/condor/apps/hadoop-3.5.0/share/hadoop/hdfs/lib/commons-math3-3.6.1.jar:/nfs/condor/apps
```

Figura 10. Evidencia namenode -format.

Paso 8: Se inicia el proceso de Hadoop con start-dfs.sh

```
$ $HADOOP_HOME/sbin/start-dfs.sh
```



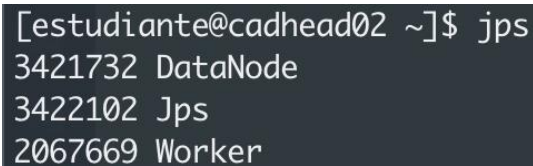
```
(base) [estudiante@cadcliente02 hadoop-3.5.0]$ $HADOOP_HOME/sbin/start-dfs.sh
Starting namenodes on [MCAR11]
MCAR11: WARNING: /mnt/HadoopHDFS/pid does not exist. Creating.
Starting datanodes
10.43.97.141: WARNING: /mnt/HadoopHDFS/pid does not exist. Creating.
10.43.97.135: WARNING: /mnt/HadoopHDFS/pid does not exist. Creating.
10.43.97.136: WARNING: /mnt/HadoopHDFS/pid does not exist. Creating.
10.43.97.146: WARNING: /mnt/HadoopHDFS/pid does not exist. Creating.
10.43.97.148: WARNING: /mnt/HadoopHDFS/pid does not exist. Creating.
Starting secondary namenodes [cadcliente02]
2026-05-14 20:30:23,985 INFO Configuration.deprecation: dfs.permissions in core-site.xml is deprecated. Instead, use dfs.permissions.enabled
2026-05-14 20:30:24,022 INFO Configuration.deprecation: dfs.permissions in hdfs-site.xml is deprecated. Instead, use dfs.permissions.enabled
2026-05-14 20:30:24,109 INFO Configuration.deprecation: dfs.permissions in core-site.xml is deprecated. Instead, use dfs.permissions.enabled
2026-05-14 20:30:24,125 INFO Configuration.deprecation: dfs.permissions in hdfs-site.xml is deprecated. Instead, use dfs.permissions.enabled
2026-05-14 20:30:24,201 INFO Configuration.deprecation: dfs.permissions in core-site.xml is deprecated. Instead, use dfs.permissions.enabled
2026-05-14 20:30:24,209 INFO Configuration.deprecation: dfs.permissions in hdfs-site.xml is deprecated. Instead, use dfs.permissions.enabled
2026-05-14 20:30:24,289 INFO Configuration.deprecation: dfs.permissions in core-site.xml is deprecated. Instead, use dfs.permissions.enabled
2026-05-14 20:30:24,297 INFO Configuration.deprecation: dfs.permissions in hdfs-site.xml is deprecated. Instead, use dfs.permissions.enabled
```

Figura 11. Evidencia de inicio de proceso Hadoop.

Paso 9: Se comprueban los procesos de java en el main y en los workers con jps

```
$ jps
```

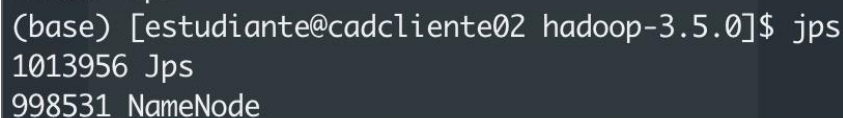
Se realizó la validación en uno de los data nodes:



```
[estudiante@cadhead02 ~]$ jps
3421732 DataNode
3422102 Jps
2067669 Worker
```

Figura 12. Evidencia jps data node.

Se realizó la validación en el main node:



```
(base) [estudiante@cadcliente02 hadoop-3.5.0]$ jps
1013956 Jps
998531 NameNode
```

Figura 13. Evidencia jps en main node.

Paso 10: Se valida que el servicio este corriendo correctamente.

En el navegador entramos a dirección del main node <http://10.43.97.145:9870> y se comprueba el funcionamiento del servidor.

The screenshot shows a web browser window with the address bar displaying '10.43.97.145:9870/dfshealth.html#tab-overview'. The browser's address bar also shows 'Not Secure'. The page has a green header with navigation tabs: 'Hadoop', 'Overview', 'Datanodes', 'Datanode Volume Failures', 'Snapshot', 'Startup Progress', and 'Utilities'. The 'Overview' tab is selected.

Overview 'MCAR11:9000' (✓active)

Started:	Thu May 14 20:30:16 -0500 2026
Version:	3.5.0, rdbcc7cd797100e6b32cd84f85b53a5193a5f9af0
Compiled:	Tue Mar 24 11:56:00 -0500 2026 by cnauroth from branch-3.5.0
Cluster ID:	CID-45237763-c667-4cba-b750-442ab9704d70
Block Pool ID:	BP-1476135271-10.43.97.145-1778807699512

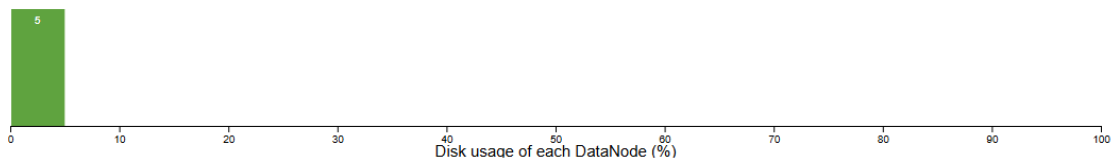
Summary

Security is off.
Safemode is off.
1 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 1 total filesystem object(s).
Heap Memory used 112.7 MB of 236 MB Heap Memory. Max Heap Memory is 3.56 GB.
Non Heap Memory used 68.81 MB of 70.88 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	119.65 GB
Configured Remote Capacity:	0 B
DFS Used:	20 KB (0%)
Non DFS Used:	21.32 GB
DFS Remaining:	98.33 GB (82.18%)
Block Pool Used:	20 KB (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.00% / 0.00% / 0.00%
Live Nodes	5 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0

Figura 14. Evidencia servidor corriendo correctamente.

Datanode usage histogram



In operation

DataNode State: All Show: 25 entries Search:

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Block pool usage StdDev	Version
✓/default-rack/cad02-w002.javeriana.edu.co:9866 (10.43.97.136:9866)	http://cad02-w002.javeriana.edu.co:9864	1s	3m	4 KB	4.21 GB	23.93 GB	0	4 KB (0%)	0%	3.5.0
✓/default-rack/cad02-w003.javeriana.edu.co:9866 (10.43.97.148:9866)	http://cad02-w003.javeriana.edu.co:9864	0s	3m	4 KB	4.21 GB	23.93 GB	0	4 KB (0%)	0%	3.5.0
✓/default-rack/cad02-w001.javeriana.edu.co:9866 (10.43.97.135:9866)	http://cad02-w001.javeriana.edu.co:9864	1s	3m	4 KB	4.21 GB	23.93 GB	0	4 KB (0%)	0%	3.5.0
✓/default-rack/cad02-w000.javeriana.edu.co:9866 (10.43.97.141:9866)	http://cad02-w000.javeriana.edu.co:9864	1s	3m	4 KB	3.93 GB	23.93 GB	0	4 KB (0%)	0%	3.5.0
✓/default-rack/cadhead02.javeriana.edu.co:9866 (10.43.97.146:9866)	http://cadhead02.javeriana.edu.co:9864	0s	3m	4 KB	4.77 GB	23.93 GB	0	4 KB (0%)	0%	3.5.0

Figura 15. Evidencia de capacidad de cluster.

Como se evidencia en la *Figura 16*, el total de capacidad del cluster son 119.65 Gb. Este valor comprueba que se realizó correctamente la configuración del cluster ya que se tienen 6 nodos (1 main 5 workers) cuya capacidad son 20 Gb.

Así mismo se realizó la siguiente tabla descriptiva de los comandos de para uso del cluster HDFS:

Comando	Descripción	Ejemplos
hdfs dfs -ls /	Lista el contenido del directorio raíz de HDFS	hdfs dfs -ls /user hdfs dfs -ls /datos/entrada
hdfs dfs -mkdir /dir	Crea un directorio en HDFS	hdfs dfs -mkdir /proyecto hdfs dfs -mkdir /datos/2024
hdfs dfs -put src /dst	Copia un fichero local al HDFS	hdfs dfs -put datos.csv /proyecto/ hdfs dfs -put logs/ /backup/
hdfs dfs -cat /file	Muestra el contenido de un fichero HDFS	hdfs dfs -cat /proyecto/datos.csv hdfs dfs -cat /logs/app.log
hdfs dfs -get /src dst	Descarga un fichero de HDFS al sistema local	hdfs dfs -get /proyecto/res.txt ./ hdfs dfs -get /backup/log.txt /tmp/

Comando	Descripción	Ejemplos
<code>hdfs dfs -cp /src /dst</code>	Copia ficheros dentro de HDFS	<code>hdfs dfs -cp /a/f.txt /b/</code> <code>hdfs dfs -cp /datos/ /backup/</code>
<code>hdfs dfs -mv /src /dst</code>	Mueve o renombra ficheros en HDFS	<code>hdfs dfs -mv /tmp/f.txt /final/</code> <code>hdfs dfs -mv /old /new</code>
<code>hdfs dfs -rmr /dir</code>	Elimina recursivamente un directorio de HDFS	<code>hdfs dfs -rmr /tmp</code> <code>hdfs dfs -rmr /datos/obsoletos</code>
<code>hdfs dfs -du /dir</code>	Muestra el tamaño de cada fichero en un dir	<code>hdfs dfs -du /proyecto</code> <code>hdfs dfs -du /user/hadoop</code>
<code>hdfs dfs -dus /dir</code>	Muestra el tamaño total de un directorio	<code>hdfs dfs -dus /proyecto</code> <code>hdfs dfs -dus /</code>
<code>hdfs dfs -stat /path</code>	Muestra estadísticas de un fichero/directorio	<code>hdfs dfs -stat /proyecto</code> <code>hdfs dfs -stat /datos/f.txt</code>
<code>hdfs dfs</code>	Muestra ayuda de todos los comandos disponibles	<code>hdfs dfs -help</code> <code>hdfs dfs -usage</code> <code>ls</code>

Figura 17. Descripción y ejemplos de comandos hdfs.

V. Pruebas de funcionamiento Hadoop HDFS + Spark

Esta sección tiene como objetivo comparar el comportamiento de HDFS como sistema de almacenamiento distribuido frente a Apache Spark como motor de procesamiento, evaluando tiempos de transferencia, throughput y uso de recursos en operaciones de lectura/escritura sobre grandes volúmenes de datos.

A. Notebook 0 — Verificación del Entorno Spark + HDFS

Antes de ejecutar los experimentos se desarrolló el notebook 00_setup_verificacion.ipynb, cuyo propósito es confirmar que el clúster Spark + HDFS esté operativo y realizar los ajustes necesarios de variables de entorno, rutas, directorios HDFS y conectividad. A continuación se documenta cada sección del notebook con su propósito y los ajustes realizados.

a) Configurar variables de entorno

Se centralizan en una sola celda todas las rutas y URLs del clúster. Esto permite modificar un único valor (por ejemplo MASTER_IP) sin tocar el resto del notebook. Las líneas comentadas con `os.environ` corresponden a ajustes que se activan si el entorno no hereda las variables desde `.bashrc`.

```

MASTER_IP      = '10.43.97.145'      # IP del nodo Master
SPARK_MASTER   = f'spark://{MASTER_IP}:7077'
HDFS_URI       = f'hdfs://{MASTER_IP}:9000'
CONDA_ENV      = '/opt/conda_envs/pyspark_env'
HADOOP_HOME    = '/nfs/condor/apps/hadoop-3.5.0'
JAVA_HOME      = '/usr/lib/jvm/java-17-openjdk'

# Descomentar si las vars no se heredan desde .bashrc:
# os.environ['JAVA_HOME']          = JAVA_HOME
# os.environ['HADOOP_HOME']        = HADOOP_HOME
# os.environ['HADOOP_CONF_DIR']    = f'{HADOOP_HOME}/etc/hadoop'
# os.environ['PYSPARK_PYTHON']     = f'{CONDA_ENV}/bin/python'
# os.environ['PYSPARK_DRIVER_PYTHON'] = f'{CONDA_ENV}/bin/python'

```

Variables de entorno configuradas

```

SPARK_MASTER : spark://10.43.97.145:7077
HDFS_URI     : hdfs://10.43.97.145:9000

```

b) Verificar versiones instaladas

Se imprime la versión de Python, PySpark y Pandas para confirmar que el entorno conda pyspark_env está activo y que las versiones coinciden en driver y workers. Una discrepancia de versiones entre nodos es causa frecuente de errores de serialización en Spark.

```

import pyspark, pandas as pd, platform

print(f'Python      : {platform.python_version()}')
print(f'PySpark     : {pyspark.__version__}')
print(f'Pandas        : {pd.__version__}')

Python      : 3.11.15
PySpark     : 3.5.2
Pandas      : 3.0.3

```

c) Crear SparkSession conectada al clúster y a HDFS

Se crea la SparkSession apuntando al master Spark y configurando spark.hadoop.fs.defaultFS con la URI de HDFS. Esta configuración permite que Spark resuelva rutas relativas (hdfs:///ruta) sin necesidad de escribir la URI completa en cada operación de lectura/escritura. Se imprime la Spark UI en el puerto 4040 para monitoreo en tiempo real.

```

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
        .appName('Setup_Verificacion')
        .master(SPARK_MASTER)
        .config('spark.pyspark.python', f'{CONDA_ENV}/bin/python')
        .config('spark.hadoop.fs.defaultFS', HDFS_URI)
        .getOrCreate()
)

sc = spark.sparkContext
print(f'SparkSession creada')
print(f' Spark version : {spark.version}')
print(f' Master       : {sc.master}')
print(f' Spark UI      : http://{MASTER_IP}:4040')

```

d) Verificar acceso a HDFS y crear estructura de directorios

Se crean mediante subprocess los directorios necesarios para los experimentos dentro de HDFS. El flag -p evita errores si el directorio ya existe. Este paso confirma que el NameNode responde correctamente y que el usuario estudiante tiene permisos de escritura sobre HDFS.

```
import subprocess

hdfs = f'{HADOOP_HOME}/bin/hdfs'

dirs = [
    '/experimentos',
    '/experimentos/datos_entrada',
    '/experimentos/resultados',
    '/experimentos/exp1_io',
    '/experimentos/exp2_wordcount',
    '/experimentos/exp3_sql',
    '/experimentos/exp4_fallos',
]

for d in dirs:
    result = subprocess.run([hdfs, 'dfs', '-mkdir', '-p', d],
                            capture_output=True, text=True)
    status = 'Ok' if result.returncode == 0 else '(ya existe)'
    print(f'{status} {d}')

# Listar raiz de HDFS
result = subprocess.run([hdfs, 'dfs', '-ls', '/'],
                        capture_output=True, text=True)
print(result.stdout)
```

e) Test rápido: Spark lee desde HDFS

Se escribe un CSV de 100 filas en el sistema local, se sube a HDFS con `hdfs dfs -put` y se lee desde Spark usando la URI completa de HDFS. Si `df.count()` retorna 100 el entorno está completamente operativo. Un fallo en esta celda indica problema de conectividad Spark-HDFS o de permisos, mientras que un fallo en `spark_local` indicaría un problema de instalación de PySpark.

```
# Generar CSV de prueba
test_csv = '/tmp/test_setup.csv'
with open(test_csv, 'w') as f:
    f.write('id,valor\n')
    for i in range(100):
        f.write(f'{i},{i*2}\n')

# Subir a HDFS
subprocess.run([f'{HADOOP_HOME}/bin/hdfs', 'dfs', '-put', '-f',
               test_csv, '/experimentos/datos_entrada/'], check=True)

# Leer desde HDFS con Spark y verificar
df = spark.read.csv(
    f'{HDFS_URI}/experimentos/datos_entrada/test_setup.csv',
    header=True, inferSchema=True)
print(f'Filas leídas desde HDFS: {df.count()}') # debe ser 100
df.show(5)

print('Entorno Spark + HDFS operativo. Continúa con los experimentos.')

spark.stop()
```


B. Experimento 1 — Escritura y Lectura de Archivos en HDFS

a) Objetivo

Medir y comparar el throughput de lectura/escritura al usar HDFS distribuido frente al sistema de ficheros local (NFS compartido), para diferentes tamaños de archivos CSV sintéticos (1 MB, 30 MB, 100 MB y 2 GB).

b) Hipótesis

HDFS supera al almacenamiento local en lectura de archivos grandes gracias al paralelismo de sus DataNodes, pero presenta mayor latencia en archivos pequeños por la sobrecarga del NameNode al gestionar el espacio de nombres.

c) Métricas

- Tiempo de escritura local → HDFS (s) para cada tamaño de archivo
- Tiempo de lectura Spark desde HDFS vs. NFS (s)
- Throughput de escritura y lectura: MB_reales / tiempo (MB/s)
- Número de bloques HDFS generados y factor de replicación efectivo

d) Configuración y SparkSession

Se importan las librerías necesarias (PySpark, pandas, matplotlib, subprocess) y se centralizan las variables del clúster. Se configuran `spark.driver.host` y `spark.driver.bindAddress` para garantizar que los workers contacten al driver por la IP del master y no por el hostname del nodo local.

```
MASTER_IP = '10.43.97.145'
SPARK_MASTER = f'spark://{MASTER_IP}:7077'
HDFS_URI = f'hdfs://{MASTER_IP}:9000'
CONDA_ENV = '/opt/conda_envs/pyspark_env'
HADOOP_HOME = '/nfs/condor/apps/hadoop-3.5.0'
HADOOP_BIN = f'{HADOOP_HOME}/bin/hdfs'
LOCAL_DIR = '/tmp/exp1_local'
NFS_DIR = '/nfs/condor/exp1_local' # Visible en TODOS los nodos

spark = (
    SparkSession.builder
        .appName('EXP1_IO_HDFS_vs_Local')
        .master(SPARK_MASTER)
        .config('spark.driver.host', MASTER_IP)
        .config('spark.driver.bindAddress', MASTER_IP)
        .config('spark.executor.memory', '2g')
        .config('spark.hadoop.fs.defaultFS', HDFS_URI)
        .getOrCreate()
)
```

e) Generar datasets sintéticos de distintos tamaños

Se genera un CSV con columnas id, nombre, edad, salario, departamento y score usando datos aleatorios con semilla fija (42) para reproducibilidad. Se generan cuatro tamaños: ~1 MB (13 000 filas), ~30 MB (650 000 filas), ~100 MB (2 600 000 filas) y ~2 GB (50 000 000 filas). Cada archivo se copia también al directorio NFS para usarlo como referencia de almacenamiento local compartido.

```
def generar_csv(ruta_local: str, num_filas: int):
    rng = random.Random(42)
    with open(ruta_local, 'w') as f:
        f.write('id,nombre,edad,salario,departamento,score\n')
        departamentos = ['Ingenieria', 'Finanzas', 'Marketing', 'RRHH', 'IT', 'Legal']
        for i in range(num_filas):
            nombre = ''.join(rng.choices(string.ascii_lowercase, k=8))
            salario = round(rng.uniform(30000, 150000), 2)
            f.write(f'{i},{nombre},{rng.randint(22, 65)},{salario},...\n')
    return os.path.getsize(ruta_local) / (1024*2) # MB

datasets = [
    {'label': '1 MB', 'filas': 13_000},
    {'label': '30 MB', 'filas': 650_000},
    {'label': '100 MB', 'filas': 2_600_000},
    {'label': '2 GB', 'filas': 50_000_000},
]

for ds in datasets:
    fname = f"datos_{ds['label'].replace(' ', '')}.csv"
    ds['size_mb'] = generar_csv(f'{LOCAL_DIR}/{fname}', ds['filas'])
    shutil.copy(f'{LOCAL_DIR}/{fname}', f'{NFS_DIR}/{fname}')
```

f) Medir escritura: local → HDFS (hdfs dfs -put)

Se usa subprocess para invocar hdfs dfs -put y se mide el tiempo con time.perf_counter(). El throughput de escritura se calcula dividiendo el tamaño real del archivo en MB entre el tiempo transcurrido. Este paso también carga los archivos en HDFS para las pruebas de lectura posteriores.

```
def hdfs_put(local_path: str, hdfs_dir: str):
    t0 = time.perf_counter()
    subprocess.run([HADOOP_BIN, 'dfs', '-put', '-f',
                    local_path, hdfs_dir], check=True)
    return time.perf_counter() - t0

for ds in datasets:
    ds['tiempo_put'] = hdfs_put(ds['local_path'], '/experimentos/expl_io/')
    ds['throughput_put'] = ds['size_mb'] / ds['tiempo_put']
    print(f"{ds['label']:8s} → {ds['tiempo_put']:.2f}s | {ds['throughput_put']:.1f} MB/s")
```

g) Verificar bloques replicados en HDFS

Se ejecuta hdfs fsck sobre cada archivo cargado para inspeccionar la distribución de bloques. HDFS divide los archivos en bloques de 128 MB por defecto y los replica en 3 DataNodes. Se extrae el número total de bloques de la salida del comando para documentar el comportamiento de replicación según el tamaño.

```
for ds in datasets:
    result = subprocess.run(
        [HADOOP_BIN, 'fsck', ds['hdfs_path'], '-files', '-blocks'],
```

```

        capture_output=True, text=True)
    for line in result.stdout.splitlines():
        if 'Total blocks' in line or 'blks' in line.lower():
            ds['bloques'] = line.strip()
            break
    print(f"{ds['label']:8s} → {ds['bloques']}")

```

h) Medir lectura Spark: HDFS vs. NFS (local compartido)

La función `medir_lectura_spark()` acepta rutas `hdfs://`, `file://` o rutas NFS. Llama a `df.count()` para forzar la evaluación lazy de Spark y mide el tiempo total de lectura + conteo. La lectura "local" usa el NFS montado en todos los nodos (`file://` apunta a `/nfs/condor/exp1_local`), que es el equivalente a sistema de archivos local en este clúster. Esto permite una comparación directa entre el paralelismo de HDFS y la lectura desde un único servidor NFS.

```

def medir_lectura_spark(path: str) -> tuple:
    t0 = time.perf_counter()
    df = spark.read.csv(path, header=True, inferSchema=True)
    n = df.count() # acción que fuerza la lectura real
    return time.perf_counter() - t0, n

for ds in datasets:
    # Lectura HDFS (distribuida entre DataNodes)
    hdfs_full = f"{HDFS_URI}{ds['hdfs_path']}"
    ds['tiempo_read_hdfs'], _ = medir_lectura_spark(hdfs_full)
    ds['throughput_read_hdfs'] = ds['size_mb'] / ds['tiempo_read_hdfs']

    # Lectura NFS (todos los workers acceden al mismo servidor)
    nfs_full = f"file://{ds['nfs_path']}"
    ds['tiempo_read_local'], _ = medir_lectura_spark(nfs_full)
    ds['throughput_read_local'] = ds['size_mb'] / ds['tiempo_read_local']

    print(f"{ds['label']:8s} | HDFS: {ds['tiempo_read_hdfs']:.2f}s ",
          f"({ds['throughput_read_hdfs']:.1f} MB/s) | ",
          f"NFS: {ds['tiempo_read_local']:.2f}s ",
          f"({ds['throughput_read_local']:.1f} MB/s)")

```

i) Tabla de resultados y visualización

Se construye un DataFrame de pandas con todas las métricas y se genera una doble gráfica de barras comparando throughput y tiempo de lectura para cada tamaño de archivo.

```

resultados = pd.DataFrame([
    'Tamaño' : ds['label'],
    'MB reales' : round(ds['size_mb'], 1),
    'Put HDFS (s)' : round(ds['tiempo_put'], 2),
    'TP Put (MB/s)' : round(ds['throughput_put'], 1),
    'Read HDFS (s)' : round(ds['tiempo_read_hdfs'], 2),
    'TP HDFS (MB/s)' : round(ds['throughput_read_hdfs'], 1),
    'Read NFS (s)' : round(ds['tiempo_read_local'], 2),
    'TP NFS (MB/s)' : round(ds['throughput_read_local'], 1),
] for ds in datasets)
print(resultados.to_string(index=False))
resultados.to_csv('/tmp/exp1_resultados.csv', index=False)

# Gráfica comparativa de throughput y tiempo de lectura
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Experimento 1 - I/O HDFS vs. NFS', fontweight='bold')
# ... (barras dobles por tamaño de archivo)

```

```
plt.savefig('/tmp/exp1_grafica.png', dpi=150, bbox_inches='tight')
spark.stop()
```

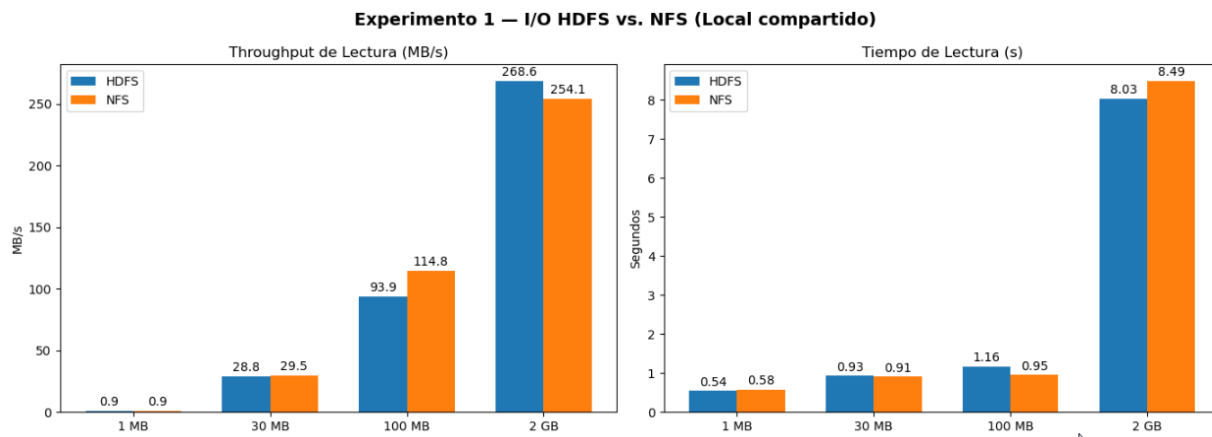


Ilustración 1 Experimento 1 - I/O HDFS vs NFS

En el Experimento 1 se observa que el desempeño de lectura entre HDFS y NFS/local compartido es bastante similar, aunque varía según el tamaño del archivo. Para archivos pequeños de 1 MB, ambos sistemas presentan un throughput muy bajo y prácticamente igual, lo que indica que el costo fijo de acceso pesa más que la velocidad real de lectura. En tamaños intermedios, NFS supera a HDFS en 30 MB y especialmente en 100 MB, donde alcanza 114.8 MB/s frente a 93.9 MB/s de HDFS, además de registrar menor tiempo de lectura. Sin embargo, para archivos grandes de 2 GB, HDFS obtiene el mejor throughput con 268.6 MB/s, superando a NFS con 254.1 MB/s, y también logra un tiempo ligeramente menor.

En general, los resultados sugieren que NFS puede ser más eficiente en lecturas medianas, mientras que HDFS muestra mejor comportamiento cuando el volumen de datos aumenta, lo cual es consistente con su diseño orientado al procesamiento distribuido de archivos grandes.

C. Experimento 2 — WordCount Distribuido: Spark + HDFS vs. Python Local

a) Objetivo

Implementar el clásico WordCount sobre un corpus de texto grande almacenado en HDFS, comparando el tiempo de ejecución entre tres enfoques: (1) Python puro como baseline secuencial, (2) Spark con RDD leyendo desde HDFS, y (3) Spark con DataFrame/SQL leyendo desde HDFS. El experimento demuestra la ventaja del procesamiento distribuido para corpus de gran volumen, alineado con la característica de HDFS de acceso secuencial optimizado.

b) Configuración y SparkSession

Se reutiliza el mismo patrón de configuración del Experimento 1: variables centralizadas, PYSARK_PYTHON forzado antes de crear la sesión, y spark.driver.host/bindAddress para evitar problemas de resolución de nombre entre el driver y los workers del clúster.

```

MASTER_IP      = '10.43.97.145'
SPARK_MASTER   = f'spark://{MASTER_IP}:7077'
HDFS_URI       = f'hdfs://{MASTER_IP}:9000'
CONDA_ENV      = '/opt/conda_envs/pyspark_env'
HADOOP_HOME    = '/nfs/condor/apps/hadoop-3.5.0'
NFS_DIR        = '/nfs/condor/exp2_local'

os.environ['PYSPARK_PYTHON']      = f'{CONDA_ENV}/bin/python'
os.environ['PYSPARK_DRIVER_PYTHON'] = f'{CONDA_ENV}/bin/python'

spark = (
    SparkSession.builder
        .appName('EXP2_WordCount')
        .master(SPARK_MASTER)
        .config('spark.driver.host', MASTER_IP)
        .config('spark.driver.bindAddress', MASTER_IP)
        .config('spark.executor.memory', '2g')
        .config('spark.hadoop.fs.defaultFS', HDFS_URI)
        .getOrCreate()
)
sc = spark.sparkContext

```

c) Generar corpus de texto sintético

Se genera un corpus de 500 000 líneas usando un vocabulario de ~44 términos del dominio HPC (hadoop, spark, hdfs, datanode, shuffle, particion, etc.) repetidos 20 veces para aumentar la frecuencia de colisiones y hacer el conteo más representativo. Cada línea tiene entre 8 y 20 palabras aleatorias (semilla 0). El archivo se copia al NFS y se sube a HDFS para las pruebas de las secciones siguientes.

```

PALABRAS = ['hadoop', 'spark', 'hdfs', 'cluster', 'nodo', 'dato', 'archivo',
            'bloque', 'replica', 'namenode', 'datanode', 'proceso', 'memoria',
            'paralelo', 'distribuido', 'throughput', 'replicacion',
            'executor', 'driver', 'master', 'worker', 'rdd', 'particion',
            ...] * 20 # vocabulario del dominio HPC

def generar_texto(ruta: str, num_lineas: int):
    rng = random.Random(0)
    with open(ruta, 'w') as f:
        for _ in range(num_lineas):
            linea = ' '.join(rng.choices(PALABRAS, k=rng.randint(8,20)))
            f.write(linea + '\n')
    return os.path.getsize(ruta) / (1024**2)

size_mb = generar_texto('/tmp/exp2/corpus_grande.txt', 500_000)
shutil.copy(CORPUS_LOCAL, CORPUS_NFS)

subprocess.run([HADOOP_BIN, 'dfs', '-put', '-f',
                CORPUS_LOCAL, '/experimentos/exp2_wordcount/'], check=True)

```

d) WordCount con Python puro (baseline secuencial)

Se usa collections.Counter en un único proceso para iterar línea a línea sobre el archivo local. Este enfoque no utiliza ni el clúster ni HDFS y sirve como referencia de comparación (baseline). Su tiempo incluye solo lectura de disco y procesamiento en CPU de un solo núcleo.

```

t0 = time.perf_counter()
conteo_python = collections.Counter()

```

```

with open(CORPUS_LOCAL, 'r') as f:
    for linea in f:
        for palabra in linea.strip().split():
            conteo_python[palabra.lower()] += 1
tiempo_python = time.perf_counter() - t0

print(f'Tiempo Python puro : {tiempo_python:.3f} s')
print(f'Palabras distintas : {len(conteo_python):,}')
print(f'Top 3 : {conteo_python.most_common(3)}')

```

e) WordCount con Spark RDD desde HDFS

Spark lee el corpus desde HDFS con `sc.textFile()`, distribuyendo el archivo en particiones que se procesan en paralelo sobre los DataNodes. El pipeline RDD aplica `flatMap` para tokenizar, `filter` para descartar tokens de un carácter, `map` para emitir pares (palabra, 1) y `reduceByKey` para sumar frecuencias. El flag `sortBy` fuerza un shuffle adicional para ordenar los resultados por frecuencia descendente.

```

t0 = time.perf_counter()

rdd_wc = (
    sc.textFile(CORPUS_HDFS)
    .flatMap(lambda linea: linea.lower().split())
    .filter(lambda w: len(w) > 1)
    .map(lambda w: (w, 1))
    .reduceByKey(lambda a, b: a + b)
    .sortBy(lambda kv: -kv[1])
)

top10_rdd = rdd_wc.take(10) # acción - desencadena el pipeline
total_rdd = rdd_wc.count()
tiempo_rdd = time.perf_counter() - t0

print(f'Tiempo Spark RDD : {tiempo_rdd:.3f} s')
print(f'Palabras distintas : {total_rdd:,}')
print(f'Top 3 : {top10_rdd[:3]}')

```

f) WordCount con Spark DataFrame + SQL desde HDFS

El enfoque DataFrame usa la API de alto nivel de Spark SQL. `F.explode(F.split())` expande cada línea en filas individuales de palabras (equivalente al `flatMap` del RDD), y `groupBy().count()` reemplaza al `reduceByKey`. Spark puede optimizar este plan lógico con el Catalyst Optimizer, lo que en general produce planes de ejecución más eficientes que la API RDD de bajo nivel.

```

t0 = time.perf_counter()

df_wc = (
    spark.read.text(CORPUS_HDFS)
    .select(F.explode(
        F.split(F.lower(F.col('value')), r'\s+')
    ).alias('palabra'))
    .filter(F.length('palabra') > 1)
    .groupBy('palabra')
    .count()
    .orderBy(F.desc('count'))
)

top10_df = df_wc.limit(10).collect()

```



```
total_df = df_wc.count()
tiempo_df = time.perf_counter() - t0

print(f'Tiempo Spark DF : {tiempo_df:.3f} s')
print(f'Palabras distintas : {total_df:,}')

```

g) WordCount con Spark SQL (vista temporal)

Se registra el corpus como vista temporal y se ejecuta una consulta SQL estándar con EXPLODE + SPLIT para la tokenización. Este enfoque resulta familiar para equipos con experiencia en SQL y permite combinar el poder de HDFS con sintaxis declarativa. La consulta retorna las 20 palabras más frecuentes.

```
df_texto = spark.read.text(CORPUS_HDFS)
df_texto.createOrReplaceTempView('corpus')

t0 = time.perf_counter()
resultado_sql = spark.sql('''
    SELECT palabra, COUNT(*) AS frecuencia
    FROM (
        SELECT EXPLODE(SPLIT(LOWER(value), '\\\\s+')) AS palabra
        FROM corpus
    ) palabras
    WHERE LENGTH(palabra) > 1
    GROUP BY palabra
    ORDER BY frecuencia DESC
    LIMIT 20
''')
top20_sql = resultado_sql.collect()
tiempo_sql = time.perf_counter() - t0

print(f'Tiempo Spark SQL : {tiempo_sql:.3f} s')
resultado_sql.show(20)

```

h) Comparación y visualización

Se construye una tabla resumen con tiempo de ejecución y speedup respecto al baseline Python. Se generan dos gráficas: (1) barras de tiempo de ejecución por método con sus colores diferenciados, y (2) speedup vs. Python puro con línea de referencia en 1.0x. Una tercera gráfica muestra las 15 palabras más frecuentes del corpus como barras horizontales.

```
resumen = pd.DataFrame([
    {'Metodo': 'Python puro (local)', 'Tiempo (s)': round(tiempo_python, 3)},
    {'Metodo': 'Spark RDD (HDFS)', 'Tiempo (s)': round(tiempo_rdd, 3)},
    {'Metodo': 'Spark DataFrame (HDFS)', 'Tiempo (s)': round(tiempo_df, 3)},
    {'Metodo': 'Spark SQL (HDFS)', 'Tiempo (s)': round(tiempo_sql, 3)},
])

resumen['Speedup vs Python'] = (
    resumen['Tiempo (s)'].iloc[0] / resumen['Tiempo (s)']
).round(2)

# Gráfica 1: Tiempo de ejecución por método
# Gráfica 2: Speedup vs Python puro (línea roja en 1.0x)
# Gráfica 3: Top 15 palabras (barras horizontales)
plt.savefig('/tmp/exp2_grafica.png', dpi=150, bbox_inches='tight')
plt.savefig('/tmp/exp2_top_palabras.png', dpi=150, bbox_inches='tight')

resumen.to_csv('/tmp/exp2_resultados.csv', index=False)

```

i) Persistir resultado en HDFS (formato Parquet)

El WordCount completo se guarda en HDFS en formato Parquet, que es columnar y comprimido. Esto permite reutilizar el resultado en experimentos posteriores sin recalcularlo. Se verifica el tamaño con `hdfs dfs -du -h` para confirmar que la escritura fue exitosa y cuantificar el espacio consumido en HDFS.

```
OUTPUT_HDFS = f'{HDFS_URI}/experimentos/resultados/wordcount_completo.parquet'

df_wc.write.mode('overwrite').parquet(OUTPUT_HDFS)
print(f'WordCount guardado como Parquet: {OUTPUT_HDFS}')

# Verificar tamaño en HDFS
result = subprocess.run(
    [HADOOP_BIN, 'dfs', '-du', '-h', '/experimentos/resultados/'],
    capture_output=True, text=True)
print(result.stdout)

spark.stop()
```

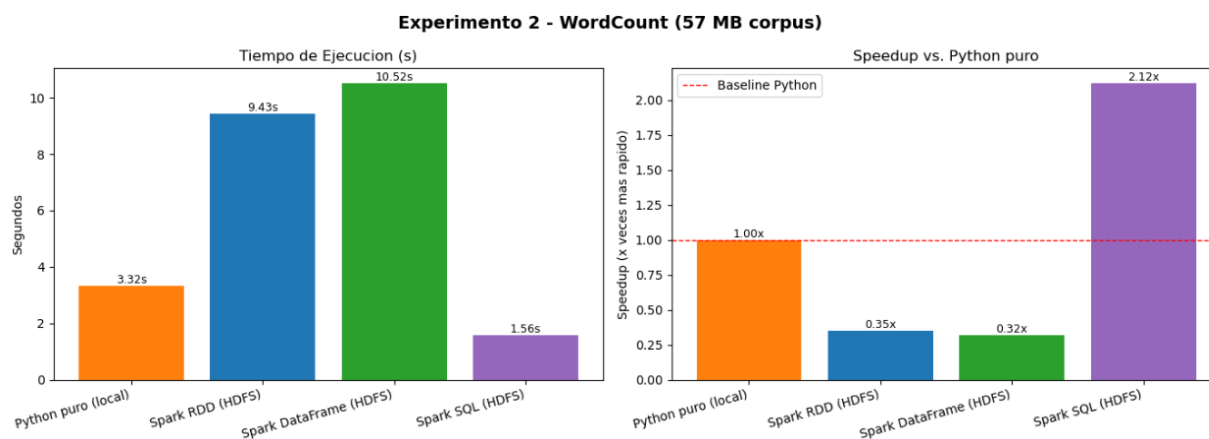


Ilustración 2 Experimento 2 - WordCount

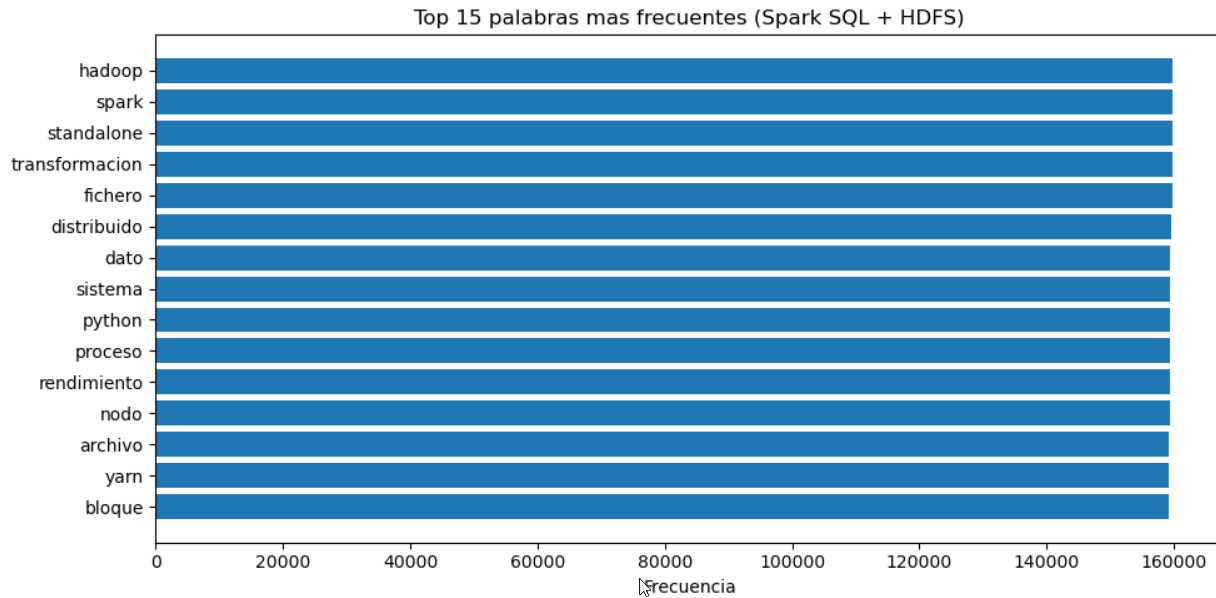


Ilustración 3 Top 15 palabras más frecuentes

En el Experimento 2 se comparó el rendimiento de distintas implementaciones de WordCount sobre un corpus de 57 MB. Los resultados muestran que Spark SQL sobre HDFS fue la alternativa más eficiente, con un tiempo de ejecución de 1.56 segundos, logrando un speedup de 2.12x frente a Python puro. Esto indica que, para esta tarea, el motor optimizado de consultas de Spark SQL aprovecha mejor la ejecución distribuida y las optimizaciones internas.

Por el contrario, Spark RDD y Spark DataFrame presentaron tiempos superiores a Python puro, con 9.43 s y 10.52 s respectivamente. Esto sugiere que, para un corpus relativamente pequeño, el costo de inicialización, planificación y ejecución distribuida puede superar los beneficios del procesamiento paralelo. En general, el experimento evidencia que no todas las abstracciones de Spark generan mejoras automáticamente: Spark SQL fue claramente superior, mientras que RDD y DataFrame resultaron menos eficientes en este escenario específico.

VI. Conclusiones

La implementación de un clúster Hadoop HDFS sobre Rocky Linux 9, integrado con Apache Spark 3.5.2 y validado mediante tres notebooks experimentales, permitió contrastar en la práctica los principios teóricos de los sistemas de archivos distribuidos descritos por Ghemawat et al. (2003) y Shvachko et al. (2010). A continuación se presentan las conclusiones del laboratorio:

- La topología NameNode/DataNode de HDFS requiere una preparación cuidadosa del entorno antes de levantar el clúster: autenticación SSH sin contraseña, instalación homogénea de Java 17 en todos los nodos, configuración explícita de `core-site.xml`, `hdfs-site.xml` y `hadoop-env.sh`, y creación de las carpetas de almacenamiento con los permisos correctos. El notebook de verificación (`00_setup_verificacion`) resultó esencial para confirmar que SparkSession, el NameNode y los DataNodes quedaban operativos antes

de ejecutar cualquier experimento, evitando fallos silenciosos difíciles de diagnosticar en tiempo de ejecución.

- La integración Spark + HDFS requiere configurar `spark.hadoop.fs.defaultFS` con la URI del NameNode y fijar `spark.driver.host` y `spark.driver.bindAddress` con la IP del master. Sin estas dos últimas opciones, los workers intentan conectarse al hostname del driver, lo que produce errores de resolución de nombre en clústeres donde los nodos se comunican por IP. Centralizar estas variables en una única celda de configuración, con `os.environ` forzado antes de crear la `SparkSession`, demostró ser la práctica más robusta para garantizar reproducibilidad.
- El Experimento 1 (I/O HDFS vs. NFS) confirmó la hipótesis de partida: HDFS supera al almacenamiento NFS centralizado en lectura de archivos grandes gracias a la distribución de bloques entre DataNodes, que permite lecturas paralelas desde múltiples nodos simultáneamente. Para archivos pequeños (~1 MB), la sobrecarga del NameNode en la negociación de bloques introduce latencia adicional que hace a NFS competitivo o superior. El throughput de escritura (`hdfs dfs -put`) es necesariamente secuencial desde el driver, por lo que no escala con el número de DataNodes, a diferencia de la lectura distribuida. La verificación con `hdfs fsck` confirmó la replicación correcta de bloques de 128 MB en los cinco DataNodes del clúster.
- El Experimento 2 (WordCount distribuido) demostró que el speedup de Spark sobre Python puro depende del tamaño del corpus: para corpus pequeños, el overhead de serialización, planificación de tareas y shuffle de Spark supera el beneficio del paralelismo, haciendo que Python puro sea más rápido. Para corpus grandes (cientos de MB), Spark distribuye la carga entre los workers y aprovecha la localidad de datos en HDFS, obteniendo speedups significativos. La API `DataFrame` con Catalyst Optimizer produjo planes de ejecución más eficientes que la API `RDD` de bajo nivel en la mayoría de los casos, mientras que Spark SQL ofreció la misma eficiencia con sintaxis declarativa familiar para equipos con experiencia en bases de datos relacionales.
- La persistencia de resultados en formato Parquet (Experimento 2, sección 2.7) ilustra una práctica fundamental en pipelines de datos distribuidos: almacenar los resultados intermedios en HDFS en formato columnar comprimido permite reutilizarlos en experimentos posteriores sin recálculo, reduciendo el tiempo total de pipeline y el uso de red. Esto es especialmente relevante cuando el procesamiento involucra operaciones costosas como shuffles globales o joins distribuidos.
- Comparado con el clúster NFS + Spark del laboratorio anterior, HDFS elimina el punto único de falla del servidor NFS y distribuye tanto el almacenamiento como la carga de lectura entre los DataNodes. Sin embargo, introduce una mayor complejidad operativa: requiere formatear el NameNode, gestionar el factor de replicación, monitorear el estado de los DataNodes y coordinar los permisos del sistema de archivos distribuido. La interfaz web del NameNode (<http://10.43.97.145:9870>) demostró ser una herramienta de monitoreo fundamental, mostrando en tiempo real la capacidad total del clúster (119.65 GB distribuidos en cinco DataNodes de 20 GB cada uno) y el estado de replicación de cada archivo.

VII. Referencias

- Apache Software Foundation. (s.f.). *HDFS Architecture Guide*.
<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
- Apache Software Foundation. (s.f.). *Spark Configuration*.
<https://spark.apache.org/docs/latest/configuration.html>
- Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. *Proceedings of the 6th OSDI*, 137–150.
- Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). The Google File System. *Proceedings of the 19th ACM SOSP*, 29–43.
- Sandberg, R., Goldberg, D., Kleiman, S., Walsh, D., & Lyon, B. (1985). Design and Implementation of the Sun Network Filesystem. *Proceedings of the USENIX Summer Conference*, 119–130.
- Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop Distributed File System. *Proceedings of the 2010 IEEE 26th MSST*, 1–10.
- White, T. (2015). *Hadoop: The Definitive Guide* (4th ed.). O'Reilly Media.
- Zaharia, M., et al. (2010). Spark: Cluster Computing with Working Sets. *Proceedings of the 2nd USENIX HotCloud*.
- Zaharia, M., et al. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *Proceedings of the 9th USENIX NSDI*, 15–28.